ARCHITECTURE AND TECHNICAL EXPLANATION

AI Candidate Assessment Platform

Backend API on Microsoft Azure

Document date: 2026-11-04
Status: phases 0 to 6 of 13 built and running in Azure
Repository: public (Azure resource names in this document are placeholders)
Audience: engineering manager review

Table of contents

1. Overview 3
2. End-to-end architecture 4
3. Architecture diagram 5
4. Key design decisions 7
5. Database interaction 8
6. Deployment 10
7. CI/CD pipeline 11
8. API architecture 13
9. Services 15
10. Layers 17
Appendix A. Known limits and open items 19

1. Overview

What the project is

The AI Candidate Assessment Platform helps companies test job candidates. A recruiter describes a job (or uploads a job-description PDF), the platform uses an AI language model to write exam questions for that job, the recruiter reviews and approves them, and, in later phases, candidates take a timed exam and are scored. This document describes the **backend**: a REST API (a web service that other programs call over HTTPS and exchange JSON data with). The existing web front end belongs to a separate project and is not covered here.

Type of application: a multi-tenant (many companies share one system, each seeing only its own data) cloud-hosted web API, built in Python with FastAPI, packaged as a container and run on Azure Container Apps.

What has been built so far (phases 0 to 6 of 13)

Capability	State
Sign-in, companies (tenants) and users, with roles (admin, recruiter)	Built and verified on real Azure
Assessments with job-description text or PDF upload (stored in Azure Blob Storage)	Built and verified on real Azure
AI question generation in the background, review, edit, regenerate and approve	Built and verified with the real Phi-4-mini-instruct model
Database with 13 tables, created automatically by migrations	Live in Azure SQL
Container image, automatic build, test, scan and release from GitHub, automatic rollback on failure	First release done; rollback drill pending
Secrets in Azure Key Vault; passwordless access using managed identities and OIDC	In place

What we are building end to end

Phases 7 to 13 are not started: participants and email invitations (7), the candidate exam with a timer (8), scoring and AI marking (9), results, overrides, export and audit (10), hardening (11), complete CI/CD gates (12) and release and handover (13). Progress is tracked in `docs/PROGRESS.md` and `docs/PROGRESS_REPORT.md`.

> **How to read this document.** Technical terms are explained when first used. Names of Azure resources are placeholders such as `<RG>` because the source repository is public. ⚠️ **TO CONFIRM** marks anything not proven during the work.

2. End-to-end architecture

At run time one small service, **assess-api**, does the work. It runs as a container in Azure Container Apps and talks to five other Azure services. A second, separate path delivers new versions of the code to it. The numbered figure in the next section shows both.

Run-time path (what happens when someone uses the API)

1. A client sends an HTTPS request to `/api/v1/...`. The Container Apps front door (the *ingress*) terminates TLS and forwards it to the container on port 8080.
2. The API checks the session cookie, finds the caller's company and role, runs the business rule and reads or writes **Azure SQL** (all data lives in the schema `assess`).
3. Uploaded job-description PDFs go to a private container in **Azure Blob Storage**; the text is extracted and stored with the assessment.
4. Question generation is slow, so the API answers immediately with `202 Accepted` and a job id. A background job inside the same process calls the **Phi-4-mini-instruct** model in **Azure AI Foundry**, validates every answer strictly, and saves the questions. The client polls the job until it is done.
5. Requests, dependency calls and logs are sent to **Application Insights**.
6. **Azure Communication Services** (email) is configured but is first used in Phase 7 (invitations); real delivery is not yet proven (⚠️ TO CONFIRM).

Secrets and identity

The application's nine secrets (database connection, signing secret, platform key, AI endpoint and key, email connection and sender, storage connection, telemetry connection) are stored in **Azure Key Vault**. The Container App holds only *references* to them and reads them through its own **managed identity** (an Azure-provided login for a program, with no password to manage). Neither the code, nor GitHub, nor the container image contains a secret.

Delivery path (how a code change reaches Azure)

Merging a change to the `main` branch starts a GitHub Actions workflow that checks, builds and releases it. GitHub proves its identity to Azure with **OIDC** (a short-lived token exchanged through Microsoft Entra ID, with no stored password), pushes the image to **Azure Container Registry**, and updates the Container App. Section 7 explains each step.

3. Architecture diagram

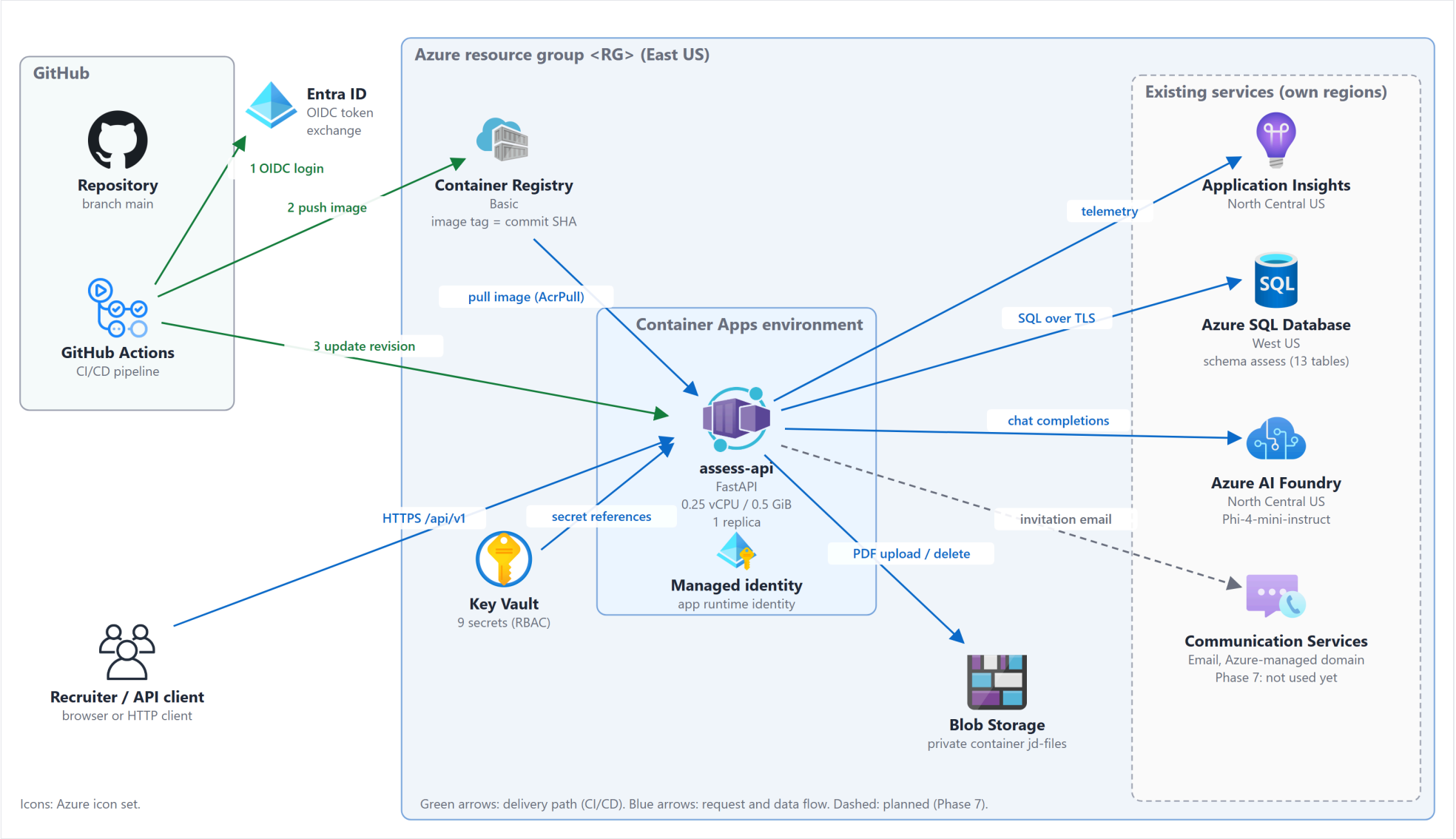


Figure 1: Azure services and how they connect. Green: delivery path. Blue: request and data flow. Dashed: planned for Phase 7. Icons from the Azure icon set; Azure AI Foundry is drawn with the generic AI services icon (⚠️ TO CONFIRM: no dedicated Foundry icon was available).

Reading the diagram

Element	What it is	Region
Resource group `<RG>`	One Azure "folder" holding all the project's resources, including some that belong to the older portal	East US
assess-api in a Container Apps environment	The API itself: 0.25 vCPU, 0.5 GiB memory, exactly one replica	East US
Container Registry, Key Vault, Blob Storage, managed identities	Created for this project	East US
Azure SQL Database	Existing serverless database shared with the portal; our tables are isolated in schema `assess`	West US
Azure AI Foundry (Phi-4-mini-instruct)	Existing AI resource and model deployment	North Central US
Application Insights	Existing telemetry resource	North Central US
Communication Services + Email	Existing communication resource; email service and Azure-managed domain created for this project	Global

4. Key design decisions

Decision	Why
Python 3.12, FastAPI, Pydantic v2	Fixed by the project's decision record (Node.js is 0 %). FastAPI gives typed request validation and automatic documentation in development.
Azure Container Apps (Consumption, 1 replica, 0.25 vCPU / 0.5 GiB)	Runs a container without managing servers and fits the $50 per month budget. One replica is deliberate: the job queue runs inside the API process, so a single instance keeps it simple (⚠️ scaling beyond one replica needs a review of the job claim logic).
Reuse the existing SQL database; isolate in schema `assess` with a least-privilege user	Avoids paying for a second database. The contained user `assess_app` owns only the schema `assess` and has no rights on the portal's tables. The database is serverless and pauses when idle, which saves money but makes the first connection after a pause slow (up to about a minute).
One repository layer for SQL Server and in-memory SQLite	The same code and tests run offline and in Azure, so laptops need no cloud services for most work.
Provider pattern with fake versions (AI, email, files)	Each external service sits behind an interface. Tests and offline runs use fakes; production uses the Azure versions. Production refuses the fakes.
Phi-4-mini-instruct as the AI model, over the OpenAI-compatible `v1` route	Already deployed and cheap. Live measurement showed it needs tolerant parsing of its answers and sometimes stalls, so the parser maps the option text to a letter and a stalled call is retried once. The strict checks (exactly four distinct options, one correct answer) were not loosened.
Background jobs in-process, stored in the database	No extra queue service to pay for or operate. Jobs survive a restart and are retried with back-off.
Key Vault references and managed identities	No secret in code, GitHub or the image. The runtime identity can only pull images and read Key Vault; the deploy identity can only push images and update this one app.
GitHub OIDC instead of a stored Azure password	Nothing to leak or rotate. GitHub's token must match a rule that names the repository and the branch or environment.
New resources in East US	Owner decision: same region as the existing portal. The database stays in West US, so each query crosses regions; measured impact is acceptable (simple request about 0.2 s from the laptop).
No Log Analytics workspace for the Container Apps environment	Saves cost. Application Insights already records requests and logs; container console logs are read on demand with the Azure CLI.
Human approval gate for every Azure change	Every script that changes Azure refuses to run until the owner writes an approval line in the provisioning plan, and runs as a dry run unless told to apply.
Production guard rails in the configuration	In production the app refuses to start with fake providers, the in-memory database, weak or development secrets, or an insecure cookie.

5. Database interaction

Connection method

The API connects to Azure SQL with **pyodbc** and Microsoft's **ODBC Driver 18**, through **SQLAlchemy 2 Core** (a Python library that builds and runs SQL). The connection is encrypted (TLS, certificate checked). The connection string comes from the secret `database-url` in Key Vault and names the least-privilege user `assess_app`. Its `Connection Timeout` value becomes the login timeout (default 30 s; 90 s is configured so a paused serverless database has time to resume).

How the data flows

1. A route calls a service (or a repository for a simple read). The caller's tenant id comes from the authenticated session, never from the request.
2. Repositories build SQL with SQLAlchemy Core. Every query on tenant data includes the tenant id.
3. Each unit of work runs in its own transaction (`Database.begin()`). Connections come from a pool: up to 10, recycled every 30 minutes, checked before use (*pre-ping*), with no overflow connections.
4. Transient Azure errors (for example a database that is resuming or briefly unavailable) are retried up to 3 times with a back-off starting at 200 ms.

Schema and migrations

All objects live in the schema `assess`. At start-up, before the API accepts traffic, a migration runner applies numbered T-SQL scripts (`0001_init`, `0002_sessions_last_seen`, `0003_jobs_questions_ai`) in order, each in its own transaction. It takes a database-wide lock (`sp_getapplock`) so two instances cannot migrate at once and records progress in `schema_migrations`. A bad connection string makes the container exit instead of serving requests.

Table	Holds	Used by
tenants, users	Companies and their staff (roles admin or recruiter; global unique, case-insensitive emails)	Phase 4
sessions, login_attempts	Login sessions (stored as hashes) and failed-login tracking for lockout	Phase 4
assessments, questions	Assessments, job description, sections, and the generated or edited questions	Phases 5 and 6
jobs	The background job queue, with status, attempts and tokens used	Phase 6
audit_log	Record of important actions	Written today by tenant, user and assessment actions; reporting in Phase 10
participants, attempts, responses, scores	Candidates, exam attempts, answers and marks	Tables exist; used from Phases 7 to 10
schema_migrations	Which migrations were applied	Migration runner

Verified on real Azure: 13 tables in `assess`, migrations 1 to 3 recorded, and no tables created in the shared `dbo` schema.

6. Deployment

The API is deployed as a container image to **Azure Container Apps**. The image is built from `api/Dockerfile` (Python 3.12 slim, Microsoft ODBC Driver 18, non-root user) on a GitHub runner and stored in **Azure Container Registry** under the tag of the full commit SHA (never `latest`), so the running version can always be traced to a commit.

Setting	Value
Size and scale	0.25 vCPU, 0.5 GiB, minimum and maximum replicas both 1, single-revision mode
Ingress	External HTTPS, target port 8080
Image pull	By the app's managed identity (role AcrPull); registry admin user is disabled
Secrets	Nine Key Vault references (container-app secret keys are limited to 20 characters, so three long names have short keys: `acs-conn`, `blob-conn`, `appi-conn`)
Settings	Production mode, SQL backend, Azure AI, email and files, model deployment name, container name, URLs (21 settings in total, nine of them secret references)
Start-up	Applies migrations, then starts the job loop and listens; the platform waits for the health check `/api/v1/health`
Shutdown	Graceful: 8 s for open requests plus up to 20 s for running jobs, inside Azure's 30 s limit
Version proof	Every response carries `x-app-version` equal to the commit SHA; the smoke test compares it
Rollback	Automatic to the previous image if the smoke test fails; manual through the `rollback.yml` workflow with an image tag (a full commit SHA)

Verified on the deployed service: health returns `{"status":"ok"}`, `x-app-version` equals the deployed commit, API documentation returns 404 in production, requests without a session return 401, and the start-up log shows migrations, configuration and the job loop starting. A repeat of all 28 live checks on the deployed API is pending (⚠️ TO CONFIRM).

7. CI/CD pipeline

CI/CD means continuous integration (automatically check every change) and continuous delivery (automatically release a verified change). It is implemented with **GitHub Actions**: YAML files in `.github/workflows/`. Each file defines a *workflow* with a trigger (`on:`), one or more *jobs* (run on a fresh Linux machine) and numbered *steps*.

Workflow files

File	Trigger	Purpose
`deploy.yml` ("CI/CD (FastAPI)")	Push to `main`, `phase/**`, `feature/**`; pull request into `main`; manual. Pushes that change only `docs/**` or `*.md` are ignored.	Check every change; on `main` release it
`nightly.yml`	Daily at 02:00 UTC; manual	Fast verification and, when deployment is enabled, an Azure smoke test
`live-e2e.yml`	Manual (inputs: base URL, highest phase, expected version, with AI, with email)	Runs the live end-to-end checks against a deployed API
`rollback.yml`	Manual (input: image tag)	Points the app back at an older image
`load-test.yml`	Manual	Small k6 load test (20 users, at most 5 minutes)

The pipeline, step by step

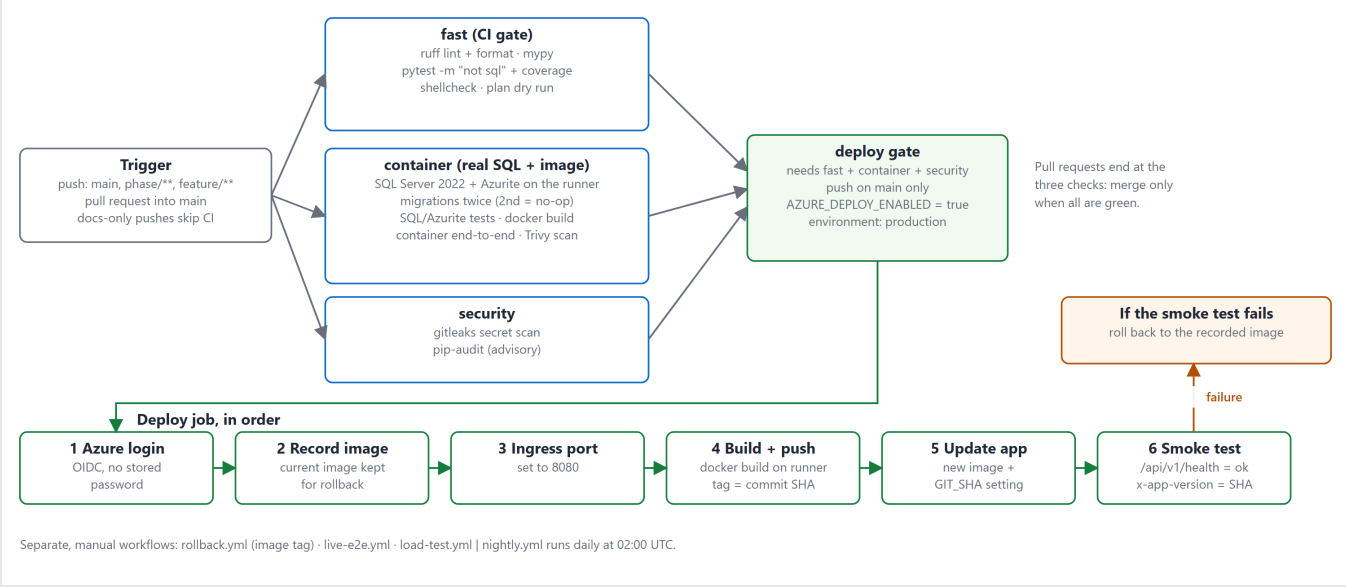


Figure 2: trigger, the three parallel checks, the deploy gate and the deploy job.

Job	What it does
fast	Installs dependencies; runs ruff (style and lint), mypy (type checking) and pytest on everything that needs no database (with coverage); runs shellcheck on the shell scripts and a dry run of the legacy provisioning plan. A few minutes in total.

container	Starts SQL Server 2022 and Azurite (a Blob Storage emulator) in Docker on the runner; creates the schema and the least-privilege user with throw-away passwords; runs the migrations twice (the second run must change nothing); runs every test against real SQL Server and Azurite (the same tests also run on SQLite); builds the image; runs the built container end to end; scans the image with Trivy (fails on fixable HIGH or CRITICAL findings).
security	gitleaks scans the repository and its history for secrets; pip-audit reports vulnerable dependencies (advisory only).
deploy	Runs only for a push to `main` when the repository variable `AZURE_DEPLOY_ENABLED` is `true`, after the three jobs above succeed, in the GitHub environment `production`. Steps: (1) sign in to Azure with OIDC; (2) record the current image for rollback; (3) set the ingress port to 8080; (4) build the image and push it to the registry with the commit SHA as tag; (5) update the app to that image and set `GIT_SHA`; (6) resolve the app address and smoke-test it: `/api/v1/health` must be ok and `x-app-version` must equal the commit SHA; (7) if any step fails, roll back to the recorded image.

How the pipeline is connected to Azure

- GitHub stores three *secrets* (the deploy identity's client id, the tenant id and the subscription id, which are identifiers rather than passwords) and the *variables* `AZURE_RESOURCE_GROUP`, `AZURE_ACR_NAME` and `AZURE_DEPLOY_ENABLED`.
- The deploy identity `<DEPLOY_IDENTITY>` trusts GitHub through **federated credentials**: rules that accept only tokens whose subject names this repository and either the `main` branch or the `production` environment. GitHub qualifies the owner and repository with numeric IDs in its subject, so the credentials must carry that exact form (a mismatch caused the first deploy attempt to fail with `AADSTS700213`).
- The identity holds only *AcrPush* on the registry and *Contributor* on the `assess-api` app.

8. API architecture

Every route is under `/api/v1`. Requests and responses are JSON. Interactive documentation is available only outside production.

Endpoints by module

Module (file)	Endpoints	Who may call
health (`routes/health.py`)	`GET /health`	Anyone
platform (`routes/platform.py`)	`POST /platform/tenants` creates a company and its first admin	Holder of the platform key; limited to 10 requests per minute per address
auth (`routes/auth.py`)	`POST /auth/login`, `POST /auth/logout`, `GET /auth/me`	Login: anyone; others: signed-in users
users (`routes/users.py`)	`GET /users`, `POST /users`, `PATCH /users/{id}`	Admins
assessments (`routes/assessments.py`)	`POST` and `GET /assessments`; `GET`, `PATCH`, `DELETE /assessments/{id}`; `POST /assessments/{id}/jd-file`; `POST /assessments/{id}/close`	Recruiters and admins
generation (`routes/generation.py`)	`POST /assessments/{id}/generate`; `GET /jobs/{id}`; `GET` and `POST /assessments/{id}/questions`; `PATCH` and `DELETE /questions/{id}`; `POST /questions/{id}/regenerate`; `POST /assessments/{id}/approve`	Recruiters and admins

Participants, candidate exam, results and audit endpoints are planned (phases 7 to 10).

Request and response flow

1. **Middleware** assigns or echoes a request id (`x-request-id`), rejects oversized bodies (1 MB, or the upload size for the PDF route) with `413`, adds security headers (`nosniff`, no framing, no referrer, HSTS, no-store caching for the API), and writes one access-log line.
2. The **route** validates the input with a Pydantic schema; bad input returns `400`.
3. A **dependency** resolves the session cookie `sid` (HttpOnly, Secure, SameSite=Lax) into the caller's id, tenant and role, or returns `401`; role checks return `403`.
4. The **service** applies business rules and uses repositories and providers.
5. The answer is returned. Every error uses one shape: `{"error": {"code": "...", "message": "...", "details": [...]}}` and always carries the request id and version headers.

The asynchronous pattern for AI generation

`POST /assessments/{id}/generate` checks the rules (job description of at least 100 words, daily limit of 20 generations per company, no approved assessment) and creates a job row, then returns `202 Accepted` with a job id. The client polls `GET /jobs/{id}` until the status is `done` or `failed`, then reads the questions.

Security rules worth knowing

- Passwords are hashed with Argon2id. After five failed logins within ten minutes, further attempts are refused until ten minutes after the last failure.
- Sessions expire after 30 minutes of inactivity or 8 hours in total; logout invalidates the session.
- Uploads: PDF or text, at most 2 MB and 200 pages; extraction has a time guard.
- Data of another company returns `404` (not "forbidden"), so its existence is not revealed.

9. Services

Application services (inside the API)

Service (`services/`)	What it does
auth	Login, session creation and validation, lockout, logout
accounts	Creates a tenant with its first admin; creates and updates users; protects the last admin
assessments	Creates, edits, closes and deletes assessments; keeps the stored job-description file in step
jd_files	Validates an uploaded file, extracts text from PDFs with `pypdf`, stores it in Blob Storage under `tenant/assessment/file`
generation	Validates a generation request, enforces daily limits and creates a job
questions	Adds, edits, deletes, regenerates and approves questions; freezes an approved assessment
rate_limit, text	Sliding-window limiter; word counting

Background job engine (`jobs/`)

A loop checks the `jobs` table every 2 seconds and runs at most 3 jobs at once, each in a worker thread. A failed job is retried up to 3 attempts with a growing delay; permanent errors fail at once with a readable message; a job left running by a crashed process is re-queued after 5 minutes. The generation handler asks for questions in batches of 5, validates each answer, drops near-duplicates and enforces the per-company daily token cap (200,000 tokens).

Providers (`providers/`)

Provider	Real implementation	Fake (offline and tests)
AI	Azure AI Foundry, OpenAI-compatible chat completions (API key in a header; retries a rate-limit answer and a stalled call once)	Scriptable, deterministic generator
Email	Azure Communication Services Email (Phase 7)	Records messages in memory
Files	Azure Blob Storage (private container)	In-memory store (Azurite in CI)

Azure services used

Service	Role in the application
Azure Container Apps	Runs the API
Azure Container Registry	Stores the application images
Azure SQL Database	Stores all application data
Azure Blob Storage	Stores uploaded job-description files
Azure AI Foundry (Phi-4-mini-instruct)	Writes the exam questions

Azure Communication Services (Email)	Will send candidate invitations (Phase 7)
Azure Key Vault	Holds the secrets
Managed identities and Microsoft Entra ID	Passwordless access for the app and for GitHub
Application Insights	Telemetry

10. Layers

The code is organised in layers so that each part has one job and can be tested on its own. Figure 3 shows the layers, and the table below states each layer's responsibility.

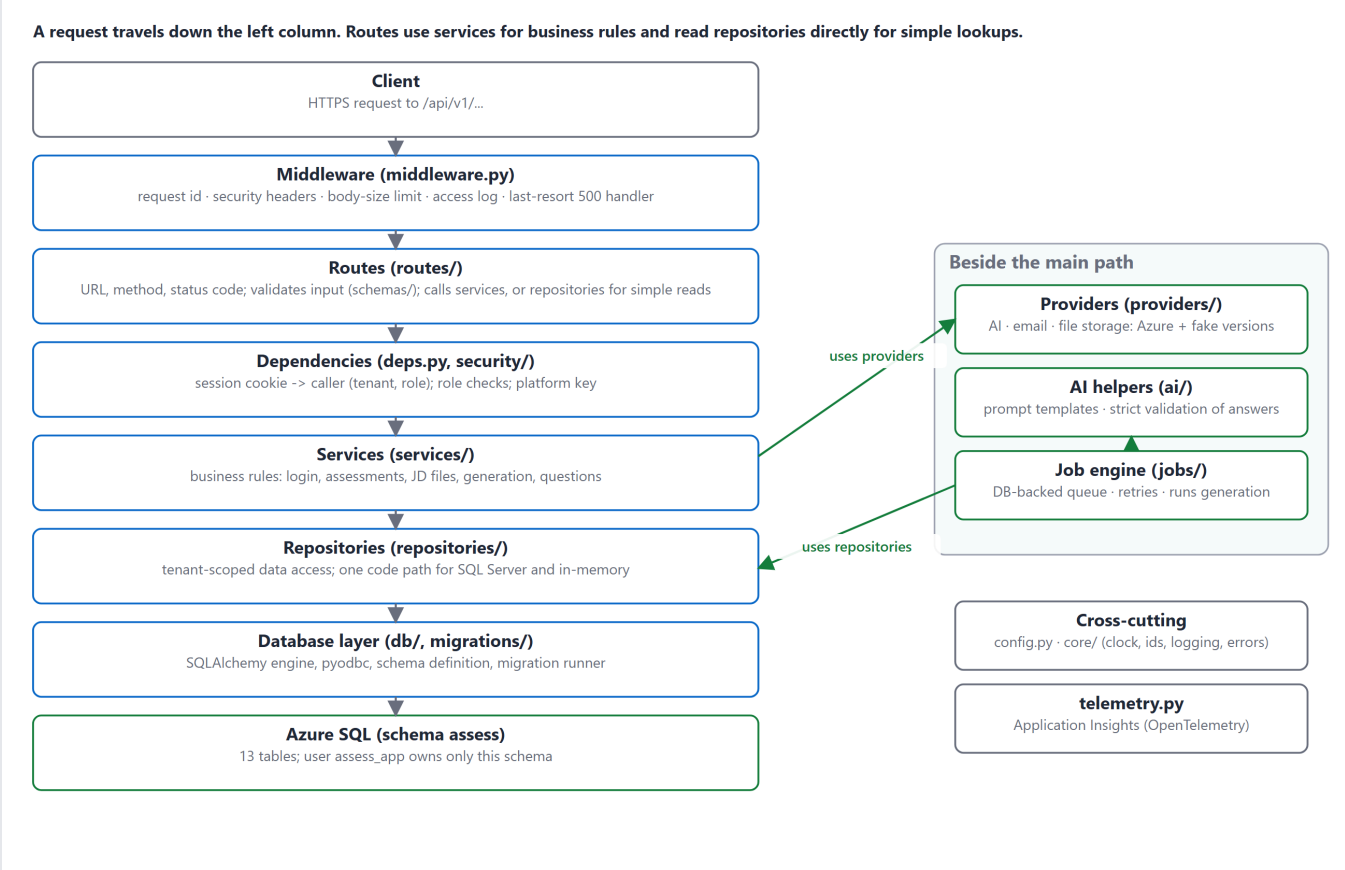


Figure 3: the layers of the API. Routes use services for business rules and read repositories directly for simple lookups.

Layer	Responsibility
Middleware (`middleware.py`)	Cross-cutting HTTP concerns: request id, security headers, body-size limit, access log, last-resort error handling
Routes (`routes/`, `schemas/`)	Define URLs, methods and status codes; validate input and shape output
Dependencies and security (`deps.py`, `security/`)	Turn the session cookie into the caller; enforce roles; password hashing and cookie signing
Services (`services/`)	Business rules
Repositories (`repositories/`)	All data access, always scoped to the caller's tenant; one implementation for SQL Server and SQLite
Database layer (`db/`, `migrations/`)	Engine, connection pooling and retry, schema definition, migration runner
Providers, AI helpers, jobs	Talk to Azure AI, email and file storage; build prompts and validate model output; run background work

Configuration and core (`config.py`, `core/`, `telemetry.py`)	Settings with production safety rules; clock, ids, logging, error types; telemetry setup

A request, end to end: "generate questions"

1. **Middleware:** request id assigned, size checked, security headers prepared.
2. **Route** `POST /assessments/{id}/generate`: validates the body.
3. **Dependency:** the session cookie becomes the caller (tenant and role); a recruiter or admin is required.
4. **Service (generation):** loads the assessment through the **repository** (tenant-scoped), checks the job description length and the daily limit, and inserts a job row. The route returns `202` with the job id.
5. **Job engine** (a few seconds later, same process): claims the job, and for each section asks the **AI provider** for a batch of questions.
6. **AI provider:** calls the Phi model, then the **validator** checks the answer strictly; an unusable answer is retried up to 3 times.
7. **Repositories** store the valid questions; the job is marked `done` with the tokens used. The client's next poll of `GET /jobs/{id}` sees `done`.

Appendix A. Known limits and open items

Item	State
Email delivery to a real mailbox	Configured; not yet proven (first task of Phase 7) ⚠️ TO CONFIRM
Actual monthly cost versus the $50 ceiling; AI token pricing	Planned allowance about $38 per month for non-AI items; not measured ⚠️ TO CONFIRM
Rollback drill and the GitHub `live-e2e` run	Pending
Full 28-check run against the deployed API	Phases 0 to 3 pass on the deployed API; the full run awaits a merged test-harness fix
Phi model behaviour	Occasionally stalls (about 3 of 20 test calls) and needs tolerant parsing; a stronger model can be deployed later by changing one setting
Single replica and in-process jobs	Intentional for the MVP; revisit before scaling out
Database region	West US while the API is in East US; acceptable now, to be re-measured under load
Teammate onboarding	No script yet to grant a teammate access; see `docs/SETUP.md` ⚠️ TO CONFIRM
Azure icon licensing	Icons come from the Azure icon set bundled with the open-source `diagrams` library ⚠️ TO CONFIRM Microsoft's current icon terms